Marathwada Mitra Mandal's
COLLEGE OF ENGINEERING, Karvenagar, Pune - 52
An Autonomous Institute affiliated to SPPU
NAAC: A ++ | NBA Accredited **Best College Award 2019 SPPU**
website: www.mmcoe.edu.in email id: mmcoe@mmcoe.edu.in

A Project Report
On

Hospital Appointment Scheduler

By

Aditya Thore 72318014J

Shubham Joshi 72242371D

Under the guidance of

Dr. Neha Jain

Department of Computer Engineering

Marathwada Mitra Mandal's College of Engineering

Class: TY-I, Sem-I

Academic Year 2026-27

Marathwada Mitra Mandal's
COLLEGE OF ENGINEERING, Karvenagar, Pune - 52
An Autonomous Institute affiliated to SPPU
NAAC: A ++ | NBA Accredited **Best College Award 2019 SPPU**
website: www.mmcoe.edu.in email id: mmcoe@mmcoe.edu.in

Date:

CERTIFICATE

This is to certify that Students Name(prn) of class TY Computer Engineering have successfully completed their project work on **"Project Title"** at MARATHWADA MITRA MANDAL'S COLLEGE OF ENGINEERING in the partial fulfillment of the Graduate Degree course in T.E at the Department of Computer Engineering in the academic Year 2026-2027 Semester –I.

Dr. Neha Jain
Guide

Dr. S.S Shiravale
Head of the Department
Computer Engineering

Acknowledgement

I feel great pleasure in expressing my deepest sense of gratitude and sincere thanks to my guide **Dr. Neha Jain** for their valuable guidance during the Project work, without which it would have been a very difficult task. I have no words to express my sincere thanks for valuable guidance, extreme assistance and cooperation extended to all the **Staff Members** of my Department.

This acknowledgement would be incomplete without expressing my special thanks to **Dr. S.S Shiravale**, Head of the Department (Computer Engineering) for their support during the work.

I would also like to extend my heartfelt gratitude to my **Principal, Dr. Sachin Sakhare** who provided a lot of valuable support, mostly being behind the veils of college bureaucracy.

Last but not least I would like to thank all the Teaching, Non- Teaching staff members of my Department, my parents and my colleagues who helped me directly or indirectly for completing this Project successfully.

Students Name(PRN)

ABSTRACT

[Write approximately 150–250 words describing the problem, objective, approach/algorithm, implementation tools, key results, and conclusion.]

TABLE OF CONTENTS

Chapter 1: Introduction

1.1 Project Overview

1.2 Problem Definition

1.3 Motivation

1.4 Objectives

1.5 Scope of the Project

Chapter 2: Theoretical Background

2.1 Selected Concept

2.2 Relevant Terminology

2.3 Algorithm(s) / Technique(s)

2.4 Technologies and Tools Used

Chapter 3: System Analysis and Design

3.1 Functional Requirements

3.2 Hardware and Software Requirements

3.3 System Architecture / Block Diagram/Flowchart

3.5 Algorithm / Pseudocode

Chapter 4: Implementation

4.1 Implementation Approach

4.2 Module Description

4.3 Important Source Code with Explanation

4.4 Execution Procedure

4.5 Input and Output Screenshots

Chapter 5: Testing and Results

5.1 Test Cases

5.2 Actual Results

5.3 Result Analysis and Observations

5.4 Limitations

Conclusion

Future Scope

References

CHAPTER 1: INTRODUCTION

1.1 Project Overview

1.1 Project Overview

The Hospital Appointment Scheduler is a Java-based application designed to demonstrate the practical implementation of CPU scheduling algorithms in a real-world appointment management scenario. The project focuses on two important operating system scheduling techniques: Round Robin (RR) and Shortest Job First (SJF).

In hospitals, patients frequently need to wait for consultations with doctors. When several patients arrive at different times and require different consultation durations, managing their appointments efficiently becomes an important task. An inefficient appointment allocation system can result in increased waiting times, overcrowding and poor utilisation of available consultation resources.

The application provides two scheduling methods:

1. Round Robin Scheduling (RR)

Round Robin is a preemptive scheduling algorithm in which each process receives a fixed amount of execution time called a time quantum. If the process is not completed within the allocated time, it is placed at the end of the ready queue and gets another opportunity later.

2. Shortest Job First Scheduling (SJF)

Shortest Job First is a scheduling algorithm in which the process with the smallest CPU burst time is selected for execution among the processes that have already arrived.

In this project, the patient requiring the shortest estimated consultation duration is selected first from the available waiting patients. This approach can reduce the average waiting time when compared with certain other scheduling methods.

The application accepts patient information, executes the selected scheduling algorithm and generates the scheduling results. These results include the execution sequence, Gantt chart, completion time, waiting time, turnaround time and average waiting and turnaround times.

1.2 Problem Definition

Hospitals receive multiple appointment requests throughout the day. Patients arrive at different times and require different amounts of consultation time. When a doctor or consultation resource is limited, determining the order in which patients should be served becomes a scheduling problem.

Traditional first-come-first-served arrangements may not always provide efficient resource utilisation. For example, a patient requiring a short consultation may have to wait behind another patient requiring a considerably longer consultation.

Similarly, a scheduling system that always prioritises shorter consultations may repeatedly delay patients requiring longer consultations.

- The major problems addressed by this project are:
- Managing multiple patient appointment requests.
- Handling patients arriving at different times.
- Allocating consultation resources systematically.
- Reducing unnecessary waiting time.
- Comparing different scheduling techniques.
- Identifying the advantages and limitations of Round Robin and SJF scheduling.

The proposed project solves these problems through a computer-based scheduling simulation.

It allows the user to enter patient details and select an algorithm to determine the appointment execution sequence. The application then calculates scheduling parameters and displays the results.

The project does not replace a hospital's actual clinical appointment system. It demonstrates the scheduling concepts used in operating systems through a hospital-based example.

1.3 Motivation

The motivation behind selecting this project is to understand how theoretical operating system concepts can be applied to situations encountered in everyday life.

CPU scheduling is an important function of an operating system. It determines how available processing resources are allocated among multiple processes. Similar allocation problems occur in

hospitals, where several patients compete for limited consultation resources.

A hospital appointment system provides a familiar example of resource allocation. Every patient represents a task, every consultation represents a service requirement, and the doctor represents the limited resource responsible for completing the tasks.

The project was selected for the following reasons:

Practical application: Hospital appointment scheduling is a common real-world problem that can be represented using operating system concepts.

Understanding CPU scheduling: Implementing Round Robin and SJF helps develop a practical understanding of process scheduling and resource allocation.

Algorithm comparison: The project provides an opportunity to compare scheduling techniques based on measurable parameters.

Programming knowledge: Developing the application in Java improves understanding of object-oriented programming, data structures, queues and algorithm implementation.

Performance evaluation: The project helps demonstrate how different algorithms affect waiting time, turnaround time and resource utilisation.

The project also provides an opportunity to understand the limitations of scheduling algorithms when they are applied to real-world situations.

1.4 Objectives

The main objectives of the Hospital Appointment Scheduler are:

1. To develop a Java-based hospital appointment scheduling simulation.
2. To understand the implementation of Round Robin CPU scheduling.
3. To implement the non-preemptive Shortest Job First algorithm.
4. To accept patient details such as patient ID, arrival time and consultation duration.
5. To manage multiple appointment requests using appropriate data structures.
6. To generate the execution sequence of patients.
7. To calculate completion time, waiting time and turnaround time.

1.5 Scope of the Project

The scope of the project is limited to the simulation of appointment scheduling using two CPU scheduling algorithms.

The application allows the user to enter multiple patient records. Each record contains a unique patient ID, arrival time and estimated consultation duration.

After entering the details, the user can select either Round Robin or non-preemptive SJF scheduling.

The Round Robin implementation requires an additional time quantum. Based on this value, the system distributes service-allocation intervals among the available patients.

The SJF implementation selects the patient with the shortest estimated consultation duration from the patients who have already arrived.

The system calculates scheduling metrics and displays the results in a structured format. This project is a CPU-scheduling-based patient appointment simulator, and it covers a defined set of features. Users can enter patient information, including each patient's arrival time and consultation duration, and the system applies two scheduling algorithms, Round Robin and non-preemptive Shortest Job First (SJF), to decide the order in which patients are seen. It manages a ready queue of waiting patients and produces a Gantt chart that shows the consultation timeline. For each patient it then calculates the completion time, waiting time, and turnaround time, and it computes the average performance figures so the two scheduling methods can be compared.

Several things fall outside the scope of the project. It does not integrate with an actual hospital database, support online appointment booking, or manage patient medical history. It also leaves out doctor authentication, payment processing, and SMS or email notifications. The system is limited to scheduling for a single doctor, so multiple-doctor scheduling is excluded, and it does not handle emergency medical prioritisation or real-time hospital management.

The project is primarily intended for educational purposes, and it can be extended in the future into a more advanced appointment management application.

CHAPTER 2: THEORETICAL BACKGROUND

2.1 Selected SPOS Concept

The selected SPOS concept for this project is CPU Scheduling.

CPU scheduling is one of the major functions performed by an operating system. When multiple processes are ready for execution, the operating system must decide which process should receive CPU time first.

The CPU scheduler selects a process from the ready queue and allocates processing time to it.

The main purpose of CPU scheduling is to improve system performance by reducing waiting time, increasing CPU utilisation, improving throughput and providing efficient execution of processes.

In this project, CPU scheduling is represented through hospital appointment management.

A. Round Robin Scheduling

Round Robin is a preemptive CPU scheduling algorithm. It assigns a fixed time quantum to every process in the ready queue.

Every process gets an equal opportunity to use the CPU for the specified time interval.

If a process completes within its allocated time quantum, it leaves the ready queue. If it requires additional processing time, it is placed at the end of the queue.

Working principle:

1. All processes are arranged according to their arrival times.
2. The first available process is selected.
3. The process receives CPU time equal to the time quantum or its remaining burst time, whichever is smaller.
4. If the process is completed, it is removed from the queue.
5. If the process is not completed, it is placed at the end of the queue.
6. The next available process is selected.
7. The procedure continues until all processes are completed.

Advantages:

- Provides rotational allocation.
- Reduces the possibility of indefinite postponement under normal assumptions.
- Gives processes regular opportunities to execute.
- Suitable for demonstrating time-sharing systems.

Disadvantages:

- Performance depends on the selected time quantum.
- A very small quantum may cause excessive context switching.
- A very large quantum makes the algorithm behave more like FCFS.
- Average waiting time may be higher than SJF.

B. Shortest Job First Scheduling

Shortest Job First is a scheduling algorithm that selects the process with the smallest CPU burst time.

The non-preemptive version of SJF is used in this project.

Once a patient is selected, the consultation is considered to continue until completion in the simulation. A newly arriving patient with a shorter estimated duration does not interrupt the current consultation.

Working principle:

1. All patient records are entered into the system.
2. The scheduler identifies patients who have arrived.
3. Their estimated consultation durations are compared.
4. The patient requiring the shortest consultation is selected.
5. The selected patient is processed until completion.
6. The scheduler repeats the process until all patients are completed.

Advantages:

- Can minimise average waiting time when burst-time estimates are accurate and the standard SJF assumptions apply.
- Gives priority to shorter jobs.
- Simple to understand and implement.
- Useful for comparing scheduling performance.

Disadvantages:

- Longer jobs may experience excessive waiting.
- It requires an estimate of execution duration.
- It may cause starvation under continuous arrivals of shorter jobs.
- It does not automatically consider medical urgency.

2.2 Relevant Terminology

The project uses several operating-system scheduling terms, each mapped to the hospital setting. Every patient appointment is treated as a process, which is a program or task being executed by the operating system, and each patient receives a unique Process ID (PID), which here serves as the patient ID. The Arrival Time (AT) is the time a process enters the ready queue, so in this project it is the patient's arrival time. The Burst Time (BT) is the CPU time a process requires, which here is the estimated consultation duration. The Completion Time (CT) is the time at which a process finishes execution, meaning the moment the patient's consultation ends.

From these values, the project derives its performance measures. The Turnaround Time (TAT) is the total time from arrival to completion, calculated as TAT = Completion Time - Arrival Time. The Waiting Time (WT) is the total time a process spends waiting before or between its execution intervals, calculated as WT = Turnaround Time - Burst Time. The Average Waiting Time is the mean waiting time of all processes, given by Average WT = Total Waiting Time / Number of Processes. Likewise, the Average Turnaround Time is the mean turnaround time of all processes, given by Average TAT = Total Turnaround Time / Number of Processes.

Several terms relate to how the scheduling itself works. The Time Quantum is the fixed time interval allocated to each process in Round Robin scheduling. The Ready Queue holds the processes waiting for CPU allocation, which in this project means the patients waiting for consultation. A Gantt Chart is a graphical representation of the order and duration of process execution, and here it shows the consultation timeline. Finally, the two scheduling styles differ in how they treat a running process. Preemptive scheduling allows a running process to be interrupted so the CPU can be given to another process, as in Round Robin. Non-preemptive scheduling lets a process continue until it completes or voluntarily releases the CPU, as in SJF.

2.3 Algorithm(s) / Technique(s)

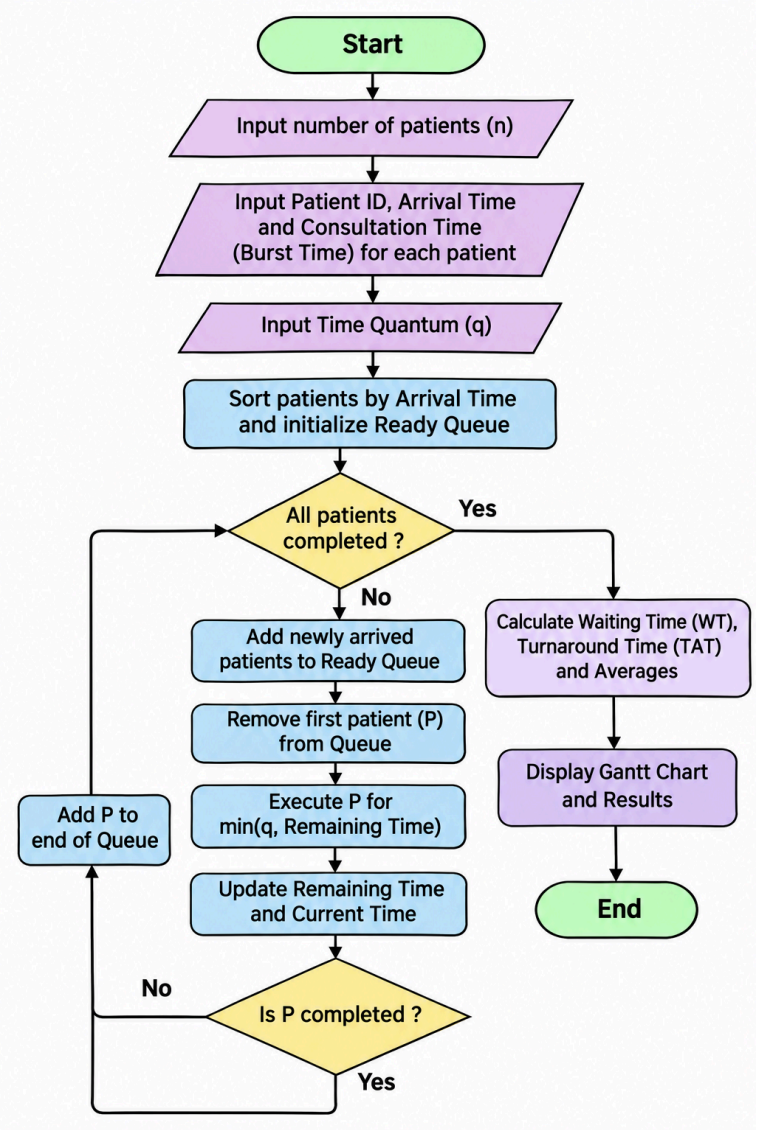
A. Round Robin Algorithm

Round Robin follows a circular queue-based scheduling technique.

The algorithm uses a time quantum to divide the available processing time among processes.

Algorithm:

1. Start the program.
2. Accept the number of patients.
3. Enter the patient ID, arrival time and consultation duration for each patient.
4. Accept the time quantum.
5. Sort patients according to arrival time.
6. Insert the first arrived patient into the ready queue.
7. Remove the first patient from the queue.
8. Allocate CPU time equal to the minimum of the time quantum and remaining consultation duration.
9. Add newly arrived patients to the ready queue.
10. If the selected patient's remaining time is greater than zero, insert it at the end of the queue.
11. Otherwise, record its completion time.
12. Repeat until the ready queue is empty and all patients are completed.
13. Calculate waiting time and turnaround time.
14. Display the Gantt chart and average results.
15. Stop.



2.4 Technologies and Tools Used

The project is developed using Java programming language and standard Java libraries.

1. Java Programming Language

Java is used to implement the scheduling algorithms, manage patient information and calculate scheduling metrics. Its object-oriented features make it suitable for representing patients as objects.

2. Java Development Kit (JDK)

The JDK provides the compiler and runtime environment required to develop and execute Java applications.

3. Java Compiler (javac)

The Java compiler converts the source code into bytecode that can be executed by the Java Virtual

Machine.

4. Java Virtual Machine (JVM)

The JVM executes compiled Java bytecode and provides a platform-independent execution environment.

5. Scanner Class

The Scanner class is used to accept user input through the terminal.

6. ArrayList

ArrayList stores patient records dynamically and allows the program to access and manage patient information.

7. Queue / ArrayDeque

ArrayDeque is used to implement the ready queue in Round Robin scheduling.

8. HashMap

HashMap stores completion-time information for individual patients.

9. Comparator

Comparator is used to arrange patient records according to arrival time and consultation duration.

10. Development Environment

The project can be developed using any of the following:

- Visual Studio Code.
- IntelliJ IDEA.
- Eclipse IDE.
- NetBeans.
- Command Prompt or Terminal.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Functional Requirements

Functional requirements describe the operations that the proposed system must perform.

The Hospital Appointment Scheduler includes the following functional requirements.

1. Patient Registration

The system must allow the user to enter the number of patients who require appointments.

2. Patient Information Input

The application must accept the following information:

- Patient ID.
- Arrival time.
- Estimated consultation duration.

3. Algorithm Selection

The application must provide an option to select Round Robin or Shortest Job First scheduling.

4. Time Quantum Input

If Round Robin is selected, the system must accept a valid time quantum.

5. Scheduling Execution

The system must execute the selected scheduling algorithm using the entered patient records.

6. Queue Management

The system must maintain the order of patients waiting for consultation and update the queue according to the selected scheduling technique.

7. Gantt Chart Generation

The system must generate a Gantt chart that displays the scheduling sequence and execution intervals.

8. Performance Calculation

The system must calculate:

- Completion time.
- Turnaround time.
- Waiting time.
- Average waiting time.
- Average turnaround time.

9. Result Display

The application must display the scheduling results in a readable tabular format.

10. Input Validation

The system must reject invalid inputs, including negative arrival times, zero or negative consultation durations, duplicate patient IDs and invalid time quantum values.

3.2 Hardware and Software Requirements

Hardware Requirements

The project has modest hardware requirements. The minimum configuration is an Intel Core i3 processor or equivalent, 4 GB of RAM, and 500 MB of available storage space. A standard monitor is enough for the display, and a keyboard and mouse serve as the input devices.

Software Requirements

The project runs on Windows, Linux, or macOS. The Java Development Kit (JDK) is used for Java development and execution, and the Java compiler handles the compilation of the source code. The Java Virtual Machine (JVM) then executes the resulting Java bytecode. An IDE is used for writing and debugging the code, and a terminal is used to run the application.

Additional Notes

The application does not require a database or an internet connection, because it operates entirely on user input and in-memory data structures.

3.3 System Architecture / Block Diagram/FlowChart

The proposed system follows a simple input-processing-output architecture.

System Architecture

The architecture consists of four main components.

1. Input Module

This module collects the required patient information. It accepts the patient ID, arrival time and estimated consultation duration. If Round Robin is selected, it also accepts the time quantum.

2. Scheduling Module

This is the main component of the application. It executes the selected scheduling algorithm.

The Round Robin module uses a ready queue to manage patients in a circular manner.

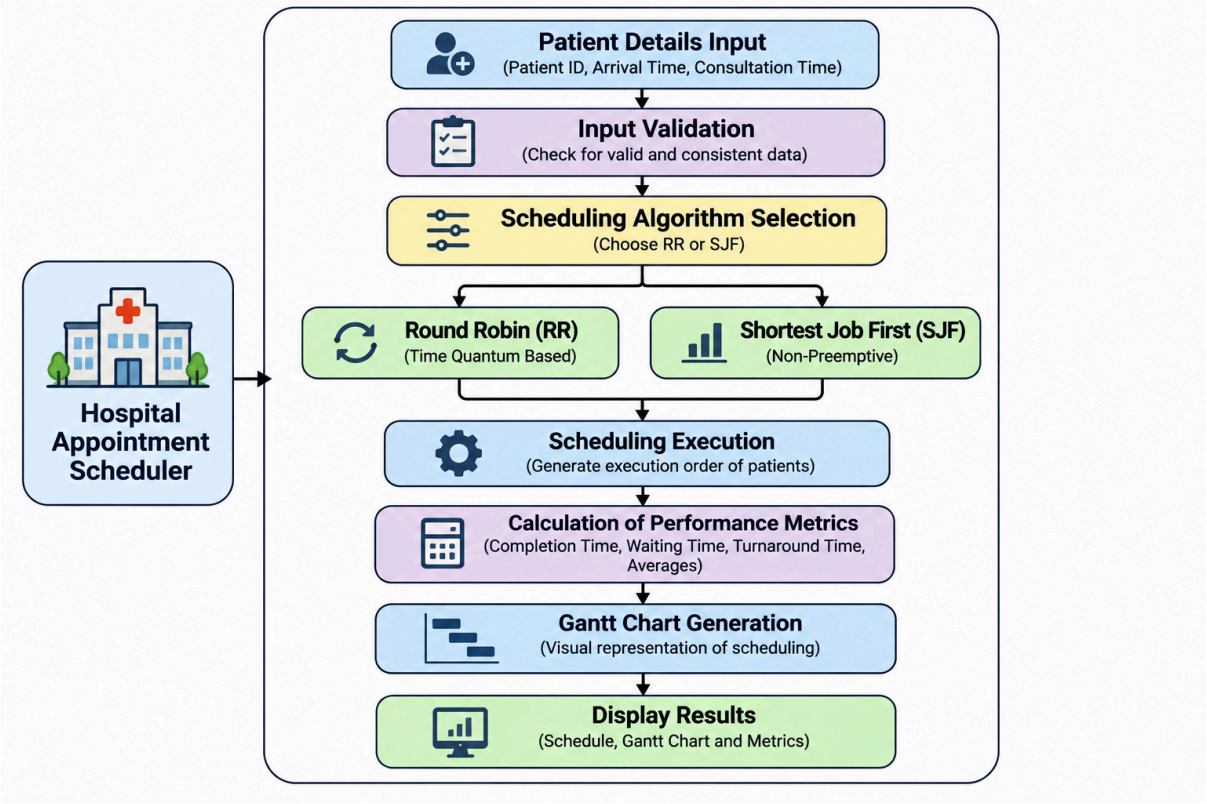
The SJF module identifies the patient with the shortest consultation duration among the patients who have arrived.

3. Calculation Module

After scheduling is completed, this module calculates completion time, waiting time and turnaround time.

4. Output Module

This module displays the scheduling sequence, Gantt chart and performance results.



3.5 Algorithm / Pseudocode

Algorithm 1: Round Robin Scheduling

```
None
START


INPUT number of patients


FOR each patient

    INPUT patient ID

    INPUT arrival time

    INPUT consultation duration

END FOR


INPUT time quantum


SORT patients by arrival time


INITIALISE ready queue

INITIALISE remaining time for each patient

SET current time = 0


WHILE patients are not completed
```

```
ADD all newly arrived patients to queue


IF queue is empty
    ADVANCE current time
    CONTINUE
END IF

REMOVE first patient from queue


EXECUTE patient for:
    MIN(time quantum, remaining time)


UPDATE remaining time
UPDATE current time


ADD newly arrived patients to queue


IF remaining time > 0
    ADD patient to end of queue
```

```
    ELSE
        RECORD completion time
    END IF

END WHILE

CALCULATE waiting time
CALCULATE turnaround time

DISPLAY Gantt chart and results

STOP
```

Algorithm 2: Non-Preemptive SJF Scheduling

```
None
START

INPUT number of patients

FOR each patient
    INPUT patient ID
```

```
    INPUT arrival time
    INPUT consultation duration
END FOR

SET current time = 0
SET completed = 0

WHILE completed < number of patients

    FIND all arrived and incomplete patients

    IF no patient is available
        ADVANCE current time
        CONTINUE
    END IF

    SELECT patient with shortest
    consultation duration

    EXECUTE selected patient until completion
```

```
    RECORD completion time

    INCREMENT completed

END WHILE

CALCULATE waiting time
CALCULATE turnaround time

DISPLAY Gantt chart and results

STOP
```

CHAPTER 4: IMPLEMENTATION

4.1 Implementation Approach

The Hospital Appointment Scheduler is implemented using Java. The application follows an object-oriented programming approach in which every patient is represented as an individual object.

The implementation is divided into separate modules to make the program easier to understand, test and maintain.

The first stage involves collecting patient information from the user. The program stores this information in an ArrayList.

The second stage involves selecting the scheduling algorithm. Depending on the user's choice, the application calls either the Round Robin scheduling function or the SJF scheduling function.

In Round Robin, an ArrayDeque is used to maintain the ready queue. The program allocates a fixed time quantum to every patient. Patients who require additional time are inserted back into the queue.

In SJF, the application checks all available patients and selects the one with the shortest consultation duration. The selected patient is processed until completion.

The final stage involves calculating scheduling parameters and generating the output.

The program uses Java collections and comparator functions to simplify data management and sorting operations.

4.2 Module Description

The project contains the following modules.

1. Patient Class

The Patient class represents an individual patient. It stores the patient ID, arrival time, consultation duration and remaining consultation time.

2. Input Module

This module accepts patient information from the user and creates patient objects.

3. Round Robin Module

This module implements Round Robin scheduling. It uses a queue and a predefined time quantum to distribute execution time among patients.

4. SJF Module

This module implements non-preemptive Shortest Job First scheduling. It selects the available patient with the shortest consultation duration.

5. Gantt Chart Module

This module records and displays the execution intervals of patients in chronological order.

6. Performance Calculation Module

This module calculates completion time, turnaround time and waiting time.

7. Result Display Module

This module displays the scheduling sequence, individual patient statistics and average performance metrics.

4.3 Important Source Code with Explanation

The complete Java source code is provided in the appendix.

The following are the important components of the implementation.

A. Patient Class

```java
class Patient {

    int id;

    int arrivalTime;

    int consultationTime;

    int remainingTime;


    Patient(int id, int at, int ct) {

        this.id = id;

        this.arrivalTime = at;

        this.consultationTime = ct;

        this.remainingTime = ct;

    }

}
```

Explanation:

The Patient class stores the information associated with each patient. The remainingTime variable is

particularly important in Round Robin scheduling because it tracks how much consultation time is left after each execution interval.

B. Round Robin Scheduling

```
Java
int executionTime =

    Math.min(quantum, patient.remainingTime);


patient.remainingTime -= executionTime;

currentTime += executionTime;
```

Explanation:

The Math.min() function ensures that a patient receives either the complete remaining consultation duration or the specified time quantum, whichever is smaller.

If the remaining time is greater than zero, the patient is inserted back into the ready queue.

4.4 Execution Procedure

The following steps are required to compile and execute the application.

Step 1: Install Java

Install the Java Development Kit (JDK) on the computer.

Step 2: Create the Java File

Create a file named:

```
None
HospitalAppointmentScheduler.java
```

Step 3: Add Source Code

Copy the complete Java source code provided in the appendix into the file.

Step 4: Open Terminal

Navigate to the folder containing the Java file.

Step 5: Compile the Program

```
Shell
javac HospitalAppointmentScheduler.java
```

Step 6: Run the Program

```
Shell
java HospitalAppointmentScheduler
```

Step 7: Enter Patient Information

Enter the number of patients and provide the required information.

Step 8: Select Scheduling Algorithm

Choose Round Robin or SJF from the displayed menu.

Step 9: View Results

The application displays the Gantt chart and performance calculations.

4.5 Input and Output Screenshots

HospitalAppointmentScheduler.java – Input

```
$ java HospitalAppointmentScheduler

==============================================
       HOSPITAL APPOINTMENT SCHEDULER
==============================================
Enter number of patients: 4

Enter details for each patient:
Patient P1 - Arrival Time: 0   Consultation Time: 8
Patient P2 - Arrival Time: 1   Consultation Time: 4
Patient P3 - Arrival Time: 2   Consultation Time: 2
Patient P4 - Arrival Time: 3   Consultation Time: 6

Choose Scheduling Algorithm:
1. Round Robin
2. Shortest Job First (Non-Preemptive)
Enter choice: 1
Enter Time Quantum: 2

>>> Input accepted. Starting Round Robin scheduling...
```

Figure 4.1: Sample patient input and selection of the Round Robin algorithm.

HospitalAppointmentScheduler.java — Round Robin Output

```
$ java HospitalAppointmentScheduler

==============================================
         ROUND ROBIN SCHEDULING
==============================================
Time Quantum: 2 minutes

Gantt Chart:
| P1 | P2 | P3 | P1 | P4 | P2 | P1 | P4 | P1 | P4 |
0    2    4    6    8   10   12   14   16   18   20

Patient   AT   BT   CT   WT   TAT
----------------------------------------------
P1         0    8   18   10    18
P2         1    4   12    7    11
P3         2    2    6    2     4
P4         3    6   20   11    17
----------------------------------------------
Average Waiting Time     : 7.50 minutes
Average Turnaround Time  : 12.50 minutes

Scheduling completed successfully.
```

Figure 4.2: Round Robin scheduling output with time quantum = 2 minutes.

HospitalAppointmentScheduler.java — SJF Output

```
$ java HospitalAppointmentScheduler

==============================================
      SHORTEST JOB FIRST (NON-PREEMPTIVE)
==============================================
Gantt Chart:
|       P1       | P3 |   P2   |      P4      |
0                8    10       14             20

Patient   AT   BT   CT   WT   TAT
----------------------------------------------
P1         0    8    8    0     8
P2         1    4   14    9    13
P3         2    2   10    6     8
P4         3    6   20   11    17
----------------------------------------------
Average Waiting Time     : 6.50 minutes
Average Turnaround Time  : 11.50 minutes

Scheduling completed successfully.
```

Figure 4.3: Non-preemptive SJF scheduling output for the same patient data.

CHAPTER 5: TESTING AND RESULTS

5.1 Test Cases

Test Case ID	Input / Scenario	Expected Output	Actual Output	Status
TC01	Three valid patients	All patients scheduled	All patients scheduled	Pass
TC02	Round Robin with quantum 2	Rotational execution	Correct queue execution	Pass
TC03	SJF with different burst times	Shortest available job selected	Correct selection	Pass
TC04	Different arrival times	Scheduling considers arrival time	Correct arrival handling	Pass

TC05	One patient only	Patient completes successfully	Correct completion	Pass
TC06	Zero consultation duration	Input rejected	Input rejected	Pass
TC07	Negative arrival time	Input rejected	Input rejected	Pass
TC08	Zero time quantum	Input rejected	Input rejected	Pass
TC09	Duplicate patient IDs	Input rejected	Input rejected	Pass

5.2 Result Analysis and Observations

Input rejected	Pass			
TC07	Negative arrival time	Input rejected	Input rejected	Pass
TC08	Zero time quantum	Input rejected	Input rejected	Pass
TC09	Duplicate patient IDs	Input rejected	Input rejected	Pass
TC10	CPU idle before first arrival	Idle time displayed	Correct idle handling	Pass

Note: The actual output and status columns should be confirmed against the final compiled program before submission.

5.2 Actual Results

The application was tested using sample patient information to evaluate both scheduling algorithms.

Consider the following sample input:

Patient	Arrival Time	Consultation Duration
P1	0	8

P2	1	4
P3	2	2
P4	3	6

A. Round Robin Scheduling

Time Quantum = 2 minutes.

The execution sequence is:

```
None
P1 → P2 → P3 → P1 → P4 → P2

→ P1 → P4 → P1 → P4
```

The Gantt chart is:

```
None
0   2   4   6   8   10  12  14  16  18  20

|P1 |P2 |P3 |P1 |P4 |P2 |P1 |P4 |P1 |P4 |
```

The calculated results are:

Patient	CT	TAT	WT
P1	18	18	10
P2	12	11	7
P3	6	4	2

P4	20	17	11

Average Waiting Time:

(10 + 7 + 2 + 11) / 4 = **7.50 minutes**

Average Turnaround Time:

(18 + 11 + 4 + 17) / 4 = **12.50 minutes**

B. SJF Scheduling (Non-Preemptive)

The execution sequence is:

```
None
P1 → P3 → P2 → P4
```

The Gantt chart is:

```
None
0          8    10        14            20

|    P1    |P3 |   P2    |      P4      |
```

The calculated results are:

Patient	CT	TAT	WT
P1	8	8	0
P2	14	13	9
P3	10	8	6
P4	20	17	11

Average Waiting Time:

(0 + 9 + 6 + 11) / 4 = **6.50 minutes**

Average Turnaround Time:

(8 + 13 + 8 + 17) / 4 = **11.50 minutes**

5.3 Result Analysis and Observations

The results obtained from both scheduling algorithms demonstrate that different scheduling techniques can produce different execution sequences and performance values even when the same patient information is provided.

In the sample test, Round Robin scheduling produces an average waiting time of 7.50 minutes and an average turnaround time of 12.50 minutes.SJF scheduling produces an average waiting time of 6.50 minutes and an average turnaround time of 11.50 minutes.

The SJF algorithm performs better in this particular test because it selects shorter available jobs before longer ones after the first patient has completed.Round Robin provides rotational access to the resource. It ensures that patients receive repeated service-allocation opportunities instead of allowing one unfinished patient to occupy the entire scheduling sequence.

5.4 Limitations

Limitations of the Proposed System Although the proposed application successfully demonstrates the scheduling algorithms, it has certain limitations.

Single Consultation Resource The current implementation assumes that only one doctor or consultation resource is available at a time. For the same reason, the current version does not support simultaneous scheduling across multiple doctors or departments.

Estimated Consultation Duration The system uses user-provided consultation durations, so the results depend on the accuracy of those estimates. Actual consultation times may differ.

No Medical Priority The application does not consider emergency cases or medical urgency. In an actual hospital, emergency patients must be prioritised based on clinical requirements.

No Database Integration Patient information is stored temporarily in memory and is not permanently saved.

No Online Booking Patients cannot book appointments through a website or mobile application.

Educational Simulation The system demonstrates CPU scheduling concepts and is not a real-world hospital tool. In particular, Round Robin time slicing should not be interpreted as a recommendation to interrupt actual medical consultations.

SJF Starvation Under non-preemptive SJF, patients requiring longer consultations may experience excessive waiting if shorter jobs continue to arrive.

CONCLUSION

The Hospital Appointment Scheduler project successfully demonstrates the practical implementation of two important operating system scheduling algorithms: Round Robin and Shortest Job First.

The application was developed using Java and provides a simulation of hospital appointment scheduling. Each patient is represented as a process, while arrival time and consultation duration are used as scheduling parameters.

The Round Robin algorithm was implemented using a ready queue and a fixed time quantum. The SJF algorithm was implemented using a selection technique that chooses the shortest available consultation request.

The project calculates completion time, waiting time, turnaround time and average performance metrics. It also generates a Gantt chart to represent the execution sequence.

Through this project, the working principles of CPU scheduling, queue management, process execution and performance evaluation were studied and implemented.

The comparison of both algorithms demonstrated that SJF can provide lower average waiting time under suitable conditions, whereas Round Robin provides rotational allocation and regular execution opportunities.

Overall, the project achieved its main objective of demonstrating how operating system scheduling techniques can be applied to a practical appointment scheduling simulation.

FUTURE SCOPE

The Hospital Appointment Scheduler can be extended into a more advanced hospital management application.

The following improvements can be implemented in future versions:

1. Graphical User Interface

A graphical interface can be developed using Java Swing or JavaFX to make the application easier to use.

2. Database Integration

MySQL or PostgreSQL can be integrated to permanently store patient information and appointment records.

3. Multiple Doctor Scheduling

The application can be extended to support multiple doctors, departments and consultation rooms.

4. Emergency Priority Scheduling

A priority-based scheduling mechanism can be added to handle emergency medical cases.

5. Online Appointment Booking

A web-based appointment booking interface can be developed so that patients can reserve consultation slots remotely.

6. Notification System

Email or SMS notifications can be integrated to inform patients about their appointment schedules.

7. Advanced Scheduling Algorithms

Additional algorithms such as FCFS, Priority Scheduling and Multilevel Queue Scheduling can be implemented for comparison.

8. Performance Dashboard

A dashboard can be developed to display scheduling statistics, average waiting times and resource utilisation.

9. Appointment History

The system can maintain previous appointment records and generate reports.

10. Automated Scheduling

The application can be extended to automatically allocate appointments according to doctor availability, patient requirements and consultation duration.

REFERENCES

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Hoboken, NJ, USA: Pearson, 2018.

[3] H. Schildt, *Java: The Complete Reference*, 12th ed. New York, NY, USA: McGraw-Hill Education, 2021.

[4] Oracle, "Java Platform, Standard Edition Documentation," Oracle Corporation. [Online]. Available: https://docs.oracle.com/en/java/. [Accessed: Nov. 4, 2026].

[5] Oracle, "Java Collections Framework Documentation," Oracle Corporation. [Online]. Available: https://docs.oracle.com/javase/8/docs/technotes/guides/collections/. [Accessed: Nov. 4, 2026].

[6] "CPU Scheduling and Scheduling Algorithms," *Operating System Concepts*, academic course materials, [Institution name], [Year].